Automation Setup

03/2026

Hello all! Welcome to a complete one-stop tutorial for setting-up your systems for the Automation Subsystem. At the end of this tutorial you would have installed all the necessary stuff that you would be needing ahead.

1 Dual-Boot

Your system needs to be dual-booted with **Ubuntu 22.04 LTS** for efficient working of all the things that are to be used ahead.

Visit this YouTube link to successfully dual boot your system

1.1 Requirements

1. Windows OS with 90-100 GBs to spare
2. An empty flash drive of atleast 16 GBs
3. Fine with your system perhaps getting ***cooked*** ;)

1.2 Things to be careful of:

Do not switch to Ubuntu 24.x version as it shall not be compatible with all things that we would be working ahead with.

2 Installing ROS2 Humble

You can follow the link provided to go through the entire official documentation available but still, necessary lines of codes have been provided ahead in this documentation for efficient installation.

ROS2 Humble Documentation

[Run the following commands on your Ubuntu 22.04 Terminal]

```
Ctrl + Alt + T                                                    //opens terminal
```

```
                    //whenever pasting/copying to/from the terminal, always press Shift
Ctrl + Shift + V
Ctrl + Shift + C
```

```
sudo apt install software-properties-common
sudo add-apt-repository universe
sudo apt update && sudo apt install curl -y

export ROS_APT_SOURCE_VERSION=$(curl -s https://api.github.com/repos/ros-
    infrastructure/ros-apt-source/releases/latest | grep -F "tag_name" | awk -F'"' '{
    print $4}')
```

```
curl -L -o /tmp/ros2-apt-source.deb "https://github.com/ros-infrastructure/ros-apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-source_${ROS_APT_SOURCE_VERSION}.$(. /etc/os-release && echo ${UBUNTU_CODENAME:-${VERSION_CODENAME}})_all.deb"
```

```
sudo dpkg -i /tmp/ros2-apt-source.deb
```

```
sudo apt update
sudo apt upgrade

sudo apt install ros-humble-desktop
```

You will always have to run the 1st command whenever you open a new terminal to execute any ROS2 Commands or instead you could run the 2nd command only once and then you would not have to run the first command ever again.

```
source /opt/ros/humble/setup.bash
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
```

To check for successful installation; run:

```
ros2
```

Expected Output:

```
usage: ros2 [-h] [--use-python-default-buffering]
            Call `ros2 <command> -h` for more detailed usage. ...

ros2 is an extensible command-line tool for ROS 2.
options:
  -h, --help            show this help message and exit
  --use-python-default-buffering
                        Do not force line buffering in stdout and instead use
                        the python default buffering, which might be affected
                        by PYTHONUNBUFFERED/-u and depends on whatever stdout
                        is interactive or not

Commands:
  action     Various action related sub-commands
  component  Various component related sub-commands
  daemon     Various daemon related sub-commands
  doctor     Check ROS setup and other potential issues
  interface  Show information about ROS interfaces
  launch     Run a launch file
  lifecycle  Various lifecycle related sub-commands
  multicast  Various multicast related sub-commands
  node       Various node related sub-commands
  param      Various param related sub-commands
  pkg        Various package related sub-commands
  plugin     Various plugin related sub-commands
  run        Run a package specific executable
```

```
  security    Various security related sub-commands
  service     Various service related sub-commands
  topic       Various topic related sub-commands
  wtf         Use `wtf` as alias to `doctor`

  Call `ros2 <command> -h` for more detailed usage.
```

3 Installing Gazebo Simulator

We are now going to install the simulator which will kind-of be replicating real world like scenarios before actually testing out your stuff in the real world.

```
sudo apt-get update
sudo apt-get install curl lsb-release gnupg
```

```
sudo curl https://packages.osrfoundation.org/gazebo.gpg --output /usr/share/keyrings/pkgs-osrf-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/pkgs-osrf-archive-keyring.gpg] https://packages.osrfoundation.org/gazebo/ubuntu-stable $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/gazebo-stable.list > /dev/null
```

```
sudo apt-get update
sudo apt-get install gz-harmonic
```

Official Gazebo Documentation

To check for successful installation; run:

```
gz
```

Expected Output:

```
The 'gz' command provides a command line interface to the Gazebo Tools.

  gz <command> [options]

List of available commands:

  help:          Print this help text.
  fuel:          Manage simulation resources.
  gui:           Launch graphical interfaces.
  launch:        Run and manage executables and plugins.
  log:           Record or playback topics.
  model:         Print information about models.
  msg:           Print information about messages.
  param:         List, get or set parameters.
  plugin:        Print information about plugins.
  sdf:           Utilities for SDF files.
```

```
  service:       Print information about services.
  sim:           Run and manage the Gazebo Simulator.
  topic:         Print information about topics.

Options:

  --force-version <VERSION>  Use a specific library version.
  --versions                 Show the available versions.
  --commands                 Show the available commands.
Use 'gz help <command>' to print help for a command.
```

In case you ever want to uninstall Gazebo; run:

```
sudo apt remove gz-harmonic && sudo apt autoremove
```

4 Getting started with TurtleBot3

We will now be installing TurtleBot3 which shall act as a dummy model to test your initials codes on and also its a great way to start learning and figuring out how things work in this field. This shall be later on replaced with an actual Artemis Drone but for now, this shall suffice.

We shall firstly create a workspace inside which the TurtleBot3 will be installed:

```
mkdir -p ~/turtlebot3_ws/src
cd ~/turtlebot3_ws/src
```

```
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git
cd ~/turtlebot3_ws && colcon build --symlink-install
```

If ever asked for a password after running a command that clones from git, put in your GitHub account's Personal Access Token [PAT]. To know how to create one, visit:

How to generate Personal Access Tokens [PATs]

To run an example world:

```
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_gazebo empty_world.launch.py
```

For more worlds, visit: TurtleBot3 Installation Documentation.

made with ❤️ by Unnat